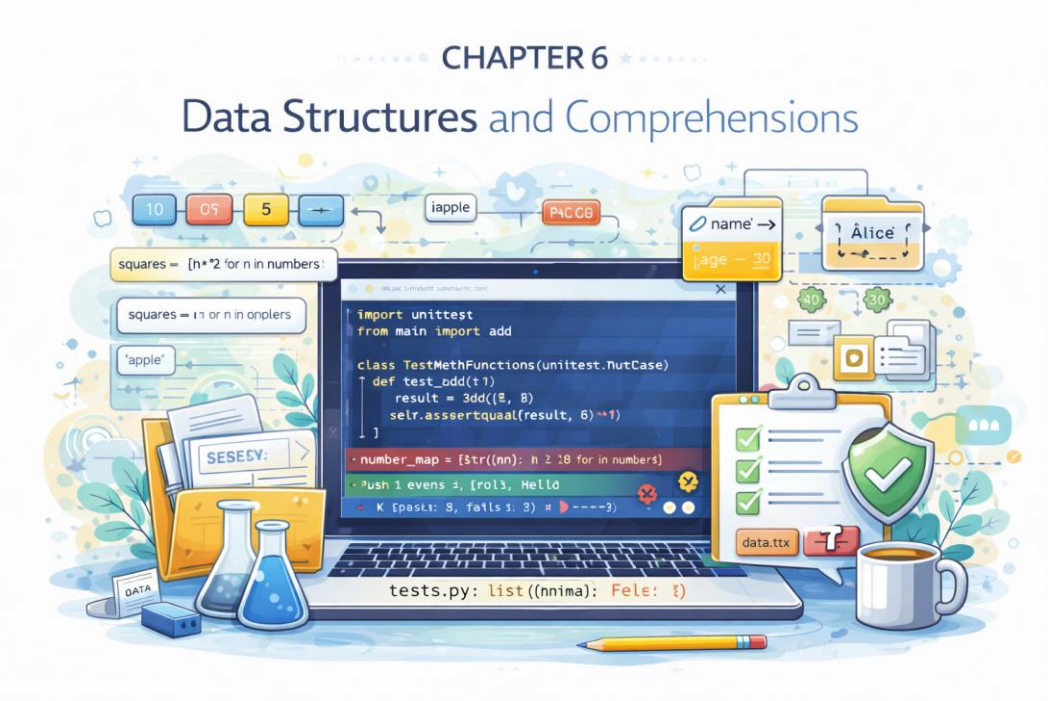

The following was generated using ChatGPT and further edited by Lucas Hartman

Chapter 6 – Data Structures and Comprehensions

As Python programs grow more complex, the way we organize, store, and process data becomes increasingly important. In earlier chapters, you worked with lists and dictionaries in simple forms—using them to hold small collections of values and to build basic application features. In this chapter, we will expand that foundation by exploring Python's full set of **built-in data structures**. Each structure—lists, dictionaries, sets, and tuples—has its own strengths, behaviors, and performance characteristics. Understanding when and why to use each one is a key part of writing clean, efficient, and expressive Python code.

Beyond working with these structures directly, Python also provides powerful tools to transform and filter data. **Comprehensions** offer a concise, readable way to build new lists, sets, and dictionaries from existing data without writing long loops. They are a core Python feature used everywhere—from data analysis and file parsing to web development and automation scripts.

In this chapter, you'll explore each major data structure in depth: what it is, how it works, its methods, and real-world situations where it applies. You'll learn to access, slice, modify, and transform collections; and compare loop-based code to comprehension-based approaches. By the end of this chapter, you'll have a solid understanding of how to store and manipulate data effectively—skills that will support every remaining chapter of this course and your final project.

6.1 Lists – A Deeper Exploration

Lists are one of the most versatile and heavily used data structures in Python. You first saw them in Chapter 1, but as your programs grow, you'll rely on lists more and more to store, organize, and manipulate collections of data. A list can represent anything from a collection of product names to daily fitness measurements to a log of interactions with a user.

A list is ordered (items maintain their position), mutable (you can change it), and can hold mixed data types (though in practice we usually keep them consistent). In this section, we'll go beyond the basics and explore indexing, slicing, methods, and two-dimensional (2D) lists.

6.1.1 Creating and Accessing Lists

You create a list using square brackets **[]** with items separated by commas:

```
fruits = ["apple", "banana", "cherry"]
```

Each item is stored at an index. Python uses zero-based indexing, meaning the first element is at index 0:

```
print(fruits[0])  # apple
print(fruits[1])  # banana
print(fruits[2])  # cherry
```

Python also lets you use negative indexes to count from the end:

```
print(fruits[-1])  # cherry  (last item)
print(fruits[-2])  # banana  (second-to-last)
```

Negative indexing is handy when you want "the most recent item" or "the last N items" without calculating lengths manually.

6.1.2 Modifying List Contents

One of the reasons lists are so popular is that they are mutable. You can update existing values, add new ones, or remove old ones.

Changing a single element:

```
fruits[1] = "blueberry"
print(fruits)  # ['apple', 'blueberry', 'cherry']
```

You can also replace multiple items at once using slice assignment:

```
numbers = [1, 2, 3, 4, 5]
numbers[1:4] = [20, 30]
print(numbers)  # [1, 20, 30, 5]
```

Here we replaced the slice **`[2, 3, 4]`** with **`[20, 30]`**. The list shrank in the process. Slice assignment allows you to "patch" ranges of data at once.

6.1.3 Slicing and Sub-lists

Slicing is a powerful feature used to extract portions of a list.

```
letters = ["a", "b", "c", "d", "e"]

print(letters[1:4])   # ['b', 'c', 'd']
print(letters[:3])    # ['a', 'b', 'c']  (start assumed 0)
print(letters[2:])    # ['c', 'd', 'e']  (go to end)
```

Slicing can also be used to reverse a list:

```
print(letters[::-1])  # ['e', 'd', 'c', 'b', 'a']
```

The general slice pattern is:

```
list_var[start:stop:step]
```

Where start and step are optional. Slices are a core tool for working with sequences in Python and appear often in real-world code.

6.1.4 Looping and Membership with Lists

To process all items in a list, you typically use a for loop. You can loop directly over the values:

```
for fruit in fruits:
    print(fruit)
```

This is ideal when you just need the values, not their index positions.

If you need to access by position (for example, to modify items in-place), you can loop over indexes:

```
for i in range(len(fruits)):
    print(f"Index {i} has {fruits[i]}")
```

The **`in`** keyword checks whether a value exists in the list:

```
if "apple" in fruits:
    print("We have apples in stock.")
```

Membership checks are common when validating user choices, avoiding duplicates, or deciding whether an item needs to be added to a collection.

6.1.5 Common List Methods

Python lists come with a variety of built-in methods to make common tasks easier.

Some of the most useful:

- **`append(x)`** – add an item to the end
- **`insert(i, x)`** – insert an item at a specific index
- **`extend(iterable)`** – add all items from another sequence
- **`remove(x)`** – remove the first occurrence of a value
- **`pop(i)`** – remove and return the item at index i (defaults to last)
- **`clear()`** – remove all items from the list
- **`sort()`** – sort the list in place
- **`reverse()`** – reverse the list in place
- **`count(x)`** – count how many times x appears
- **`index(x)`** – find the index of x (raises an error if missing)

Example:

```
inventory = ["pen", "pencil"]
inventory.append("marker")
inventory.extend(["notebook", "eraser"])

print(inventory)
# ['pen', 'pencil', 'marker', 'notebook', 'eraser']
```

These methods let you treat a list like a flexible container—one that can grow, shrink, and be reorganized as your program runs.

6.1.6 Concatenating and Repeating Lists

You can combine two lists using the **+** operator:

```
a = [1, 2, 3]
b = [4, 5]
combined = a + b
print(combined)  # [1, 2, 3, 4, 5]
```

You can repeat a list using the ***** operator:

```
greeting = ["Hello"]
triple = greeting * 3
print(triple)  # ['Hello', 'Hello', 'Hello']
```

While concatenation and repetition are convenient, be mindful of performance when dealing with extremely large lists.

6.1.7 Two-Dimensional Lists (2D Lists) – Tables of Data

A two-dimensional list (2D list) is simply a list whose elements are themselves lists. This structure is a natural fit for table-like data, such as rows and columns in a spreadsheet.

If you need to loop the entire 2D list, you will need to use a set of nested for loops.

```
expenses = [
    [1854.32, 2991.87],
    [1023.45, 2210.78, 2876.14],
    [1345.99, 1987.55, 2100.10, 2999.88],
    [1765.43, 2440.12]
]

total = 0
for r in range(len(expenses)):
    for c in range(len(expenses[r])):
        total += expenses[r][c]
```

Or if using a for each loop it would look something like this.

```
total = 0
for row in expenses:
    for column in row:
        total += column
```

2D lists can also be more rectangular and you may know what each internal list contains. For example, suppose you have an inventory table where each row stores:

- item name
- quantity in stock
- price per unit

We can represent each row as a list and the whole table as a list of rows:

```
inventory_table = [
    ["Vacuum Bags", 50, 12.99],
    ["Extension Hose", 20, 24.50],
    ["Filter Pack", 35, 9.75]
]
```

Here:

- inventory_table[0] is ["Vacuum Bags", 50, 12.99]
- inventory_table[0][0] is "Vacuum Bags"
- inventory_table[0][1] is 50
- inventory_table[0][2] is 12.99

Looping by Index When Dimensions Are Known

If you know each row has exactly 3 columns (name, quantity, price), you can also loop by index:

```
rows = len(inventory_table)

for r in range(rows):
    name = inventory_table[r][0]
    qty = inventory_table[r][1]
    price = inventory_table[r][2]
    print(f"{name}: {qty} units at ${price:.2f}")
```

Using indexes can be helpful when you need the row number (for example, to show line numbers on reports or to update a specific row by index).

Working with 2D lists gives you a good stepping stone toward more advanced data structures and eventually working with files, databases, or spreadsheet-like data.

Note: In a larger application, it's often better to represent each row as an object (like a Product class with name, quantity, and price attributes). But for small programs or early prototypes, 2D lists work well and help you practice thinking in rows and columns.

6.2 Tuples – Ordered, Immutable Sequences

Tuples are closely related to lists: they are ordered sequences of items. The key difference is that tuples are immutable—once created, they cannot be changed. This immutability makes tuples safer to use when you want to guarantee that a collection of values will stay the same.

Common uses for tuples:

- representing coordinates like (x, y)
- representing fixed groups like RGB colors (red, green, blue)
- returning multiple values from functions
- using them as dictionary keys (lists cannot be dictionary keys, but tuples can)

6.2.1 Creating Tuples

You create a tuple using parentheses **()**:

```
point = (3, 4)
color = ("red", 255, 0)
```

You can access items just like in a list:

```
print(point[0])  # 3
print(point[1])  # 4
```

You can also slice tuples:

```
coords = (10, 20, 30, 40)
print(coords[1:3])   # (20, 30)
```

Single-Value Tuples

One small "gotcha" is single-value tuples. The comma is what makes it a tuple, not the parentheses:

```
x = (5)    # this is just the integer 5
x = (5,)   # this is a tuple containing 5
```

6.2.2 When to Choose Tuples Over Lists

Use a tuple when:

- you want to signal "this should not change"
- you need to use the collection as a dictionary key
- you are packaging multiple return values together

For example, a function that calculates statistics might return a tuple:

```
def get_min_max(values):
    return (min(values), max(values))

low, high = get_min_max([5, 9, 2, 8])
```

Here, returning a tuple clearly expresses that the result is a fixed pair of values.

6.3 Dictionaries – Keyed Data and Fast Lookups

While lists are driven by position (index 0, 1, 2, …), dictionaries are driven by keys. A dictionary allows you to look up values using meaningful names or IDs instead of numeric positions. This is perfect for representing objects like students, customers, configuration settings, or anything where you want to label data with names.

A dictionary:

- is a collection of key–value pairs
- has unique keys (you can't have the same key twice)
- provides fast lookups by key

6.3.1 Creating and Accessing Dictionaries

You create a dictionary using curly braces {} with key: value pairs:

```
student = {
    "name": "Ava",
    "age": 20,
    "major": "Computer Science"
}
```

To access values, use the **key** inside square brackets:

```
print(student["name"])    # Ava
print(student["age"])     # 20
```

If a **key** does not exist, Python raises a **KeyError**:

```
# student["gpa"]  # KeyError: 'gpa'
```

6.3.2 Using .get() vs Direct Access

The **.get()** method safely retrieves a **value**. If the **key** is missing, it returns **None** by default or a value you provide:

```
gpa = student.get("gpa")          # None
gpa = student.get("gpa", 0.0)     # 0.0
```

By contrast, **student["gpa"]** would raise an exception if the key is missing. Use **.get()** when the key may or may not exist.

6.3.3 Adding and Updating Key–Value Pairs

You can add new entries or change existing ones with simple assignment:

```
student["gpa"] = 3.8          # add new key
student["major"] = "Software Development"
                                      # update existing
```

6.3.4 Deleting Keys from a Dictionary

Sometimes information needs to be removed—for example, when clearing a temporary value or removing a field you no longer want to store.

Using **del**:

```
del student["gpa"]
```

del removes the key entirely, but raises a **KeyError** if the key doesn't exist. Use caution when you're unsure the key exists.

Using **.pop()**:

```
gpa = student.pop("gpa", None)
```

.pop() removes the **key** and returns its value. If the key is missing, it returns the default value instead of raising an exception.

6.3.5 Helpful Dictionary Methods

Dictionaries provide helpful "view" objects of dictionary contents. These views also reflect any changes made to the original dictionary. These are objects that are read-only to access the content.

- keys() returns a view object containing all the keys
- values() returns a view object containing all the values
- items() returns a view object containing all the key-value pairs in the dictionary presented as a tuple (key, value).

```
print(student.keys())     # dict_keys([...])
print(student.values())   # dict_values([...])
print(student.items())    # dict_items([...])
```

These can also be converted to a list using the list() function discussed later in the chapter.

You will see us using these below in different types of loops.

6.3.6 Looping Through Dictionaries

Here are several useful looping patterns:

Looping over keys (default):

```
for key in student:
    print(key, "=>", student[key])
```

Looping over keys explicitly:

```
for key in student.keys():
    print("Key:", key)
```

Looping over values:

```
for value in student.values():
    print("Value:", value)
```

Looping over key–value pairs:

```
for key, value in student.items():
    print(f"{key}: {value}")
```

6.3.7 Combining Dictionaries

To merge two dictionaries in Python 3.9+, use the **|** operator:

```
defaults = {"theme": "light", "autosave": True}
user_settings = {"theme": "dark"}

final_settings = defaults | user_settings
print(final_settings)
# {'theme': 'dark', 'autosave': True}
```

6.3.8 The dict() Constructor

The **`dict()`** function gives you another way to build dictionaries:

```
user = dict(name="Lucas", role="Instructor", active=True)
```

You can also pass a list of key–value pairs:

```
pairs = [("name", "Ava"), ("age", 20)]
student2 = dict(pairs)
```

6.3.9 Using __dict__ with Custom Objects

Internally, Python stores an object's attributes in a dictionary named **__dict__**. You can access it directly:

```
class Product:
    def __init__(self, name: str, price: float):
        self.name = name
        self.price = price

p = Product("Vacuum", 199.99)
print(p.__dict__)  # {'name': 'Vacuum', 'price': 199.99}
```

This makes it very easy to convert objects into dictionaries for JSON serialization, database storage, or debugging.

In the next chapter we will discuss using this to write out JSON data. We also will talk about overriding it if we would like to hide certain data to keep concealed.

6.4 Sets – Unique, Unordered Collections

Sets are collections that store unique items without caring about order. You can think of a set as "a bag of distinct values." They are excellent when you want to avoid duplicates, check membership quickly, or compare groups of items.

Real-world examples where sets are useful:

- tracking which customers have placed an order this month
- cleaning up a list of emails so each address appears only once
- determining which products are in both Warehouse A and Warehouse B

6.4.1 Creating Sets

You create a set using curly braces **{}** with comma-separated values:

```
colors = {"red", "green", "blue"}
```

This looks similar to a dictionary, but there are no key–value pairs—just values.

One important detail:

```
empty = {}        # This is an empty dictionary, NOT a set
empty_set = set()  # This creates an empty set
```

Because **{}** is reserved for empty dictionaries, you must use `set()` for an empty set.

6.4.2 Why Sets Matter

Sets are particularly powerful for operations that sound like math:

"What items are in both of these groups?"

"Which items are only in this group?"

"What items are different between these two?"

These map directly to set operations like intersection, difference, and symmetric difference.

6.4.3 Set Operations and Their Symbols

Given two sets A and B:

- Union – all items that appear in A or B (or both)
- Intersection – items that appear in both A and B
- Difference – items in A that are not in B
- Symmetric difference – items in either A or B but not in both

Python provides both methods and operators:

Union: **`A.union(B)`** or **`A | B`**

Intersection: **`A.intersection(B)`** or **`A & B`**

Difference: **`A.difference(B)`** or `A - B`

Symmetric difference: **`A.symmetric_difference(B)`** or **`A ^ B`**

Example with inventory between warehouses:

```
warehouse_a = {"filter", "bag", "hose"}
warehouse_b = {"bag", "belt", "brush"}

both = warehouse_a & warehouse_b
print("Both carry:", both)  # {'bag'}

all_items = warehouse_a | warehouse_b
print("All unique items:", all_items)

only_a = warehouse_a - warehouse_b
print("Only in A:", only_a)

different = warehouse_a ^ warehouse_b
print("Not shared:", different)
```

These operations are extremely fast and make sets a powerful tool for comparing and cleaning up data.

6.4.4 Sets with custom objects

You can use sets to again have unique objects. For example, we don't want two of the same people in a system where we have an id, name, street address, etc.

For this you need to define the **__eq__** and **__hash__** methods.

We have already seen **__eq__** where we defined what attributes on an object need to be compared to see if objects are equal.

If you want to use a custom object in a set OR as the key in a dictionary you need to also override the **__hash__** method. The **__hash__** method returns a number that represents the object. Python uses this number to place the object into a "bucket" inside the set for extremely fast look up. To create the hash you use the **hash()** function and send it the same attributes you are comparing in the **__eq__** method.

```
class Product:
    def __init__(self, name, price):
        self.name = name
        self.price = price
    def __eq__(self, other):
        return isinstance(other, Product)
        and self.name == other.name
        and self.price == other.price
        )
    def __hash__(self):
        return hash((self.name, self.price))
```

When comparing multiple attributes put them in a tuple. Above code we have them with parentheses – the second set are the parentheses for the **`hash()`** function

Then creating two objects with same values will register as equal and putting in a set will yield only keeping one.

```
p1 = Product("Bag", 9.99)
p2 = Product("Bag", 9.99)

items = {p1, p2}
print(len(items))   # 1, because p1 and p2 are considered equal
```

6.5 Converting Between Data Structures

Python's constructor functions—**`list()`**, **`dict()`**, **`set()`**, and **`tuple()`**—make it easy to convert between data structures. This might seem like a small convenience, but data conversion is extremely common in real programs. The same information may need to behave like:

- a list (for ordering and indexing)
- a set (for uniqueness and comparisons)
- a dictionary (for keyed lookup)

Being able to switch between these forms helps you write flexible and powerful code.

6.5.1 Converting to a List – list()

`list()` takes any iterable (including sets, tuples, and dictionary views) and returns a list:

```
s = {"apple", "banana", "cherry"}
fruit_list = list(s)
```

You might do this to:

- index into data that started as a set
- sort data that came from another structure
- store dictionary keys in a list for further processing:

```
student = {"name": "Ava", "age": 20}
keys = list(student.keys())
values = list(student.values())
```

6.5.2 Converting to a Tuple – tuple()

`tuple()` creates an immutable sequence from an iterable:

```
nums = [1, 2, 3]
nums_tuple = tuple(nums)
```

This is useful when you want to "lock down" the data so it cannot be changed accidentally, or when you need a hashable (immutable) structure to use as a dictionary key.

6.5.3 Converting to a Set – set()

`set()` is frequently used to remove duplicates and to prepare for fast membership checks:

```
items = ["pen", "pen", "pencil"]
unique_items = set(items)  # {'pen', 'pencil'}
```

You can always convert it back to a list afterward:

```
items_list = list(unique_items)
```

Combining these steps is a common pattern for cleaning data.

6.5.4 Converting to a Dictionary – dict()

`dict()` can construct a dictionary from:

- another dictionary
- a sequence of (key, value) pairs
- keyword arguments

```
pairs = [("name", "Lucas"), ("role", "Instructor")]
info = dict(pairs)

settings = dict(debug=True, version="1.0")
```

It also works nicely with **`__dict__`** when exporting objects:

```
record = dict(p.__dict__)
```

Understanding these conversion functions lets you shape your data to fit the task at hand.

Note on naming conventions of variables. While they will work in PyCharm you will get a warning about using “list” “dict” “set” “tuple” as variable names due to possible collisions with these functions and the structure data types. I would make sure to have variables named something else, or at least add in extra nouns name_list, people_dict, etc.

6.6 Built-In Functions for Working with Collections

Python includes several built-in functions that work across lists, tuples, sets, and other iterable objects. These functions help you summarize, sort, and evaluate collections in compact and readable ways.

6.6.1 len() – Length of a Collection

`len()` returns how many items are in a collection:

```
names = ["Ava", "Noah", "Mia"]
print(len(names))  # 3
```

You can use **`len()`** to:

- control loops (range(len(list)))
- validate input (e.g., ensure a password is at least 8 characters)
- check whether a collection is empty (len(collection) == 0)

6.6.2 sum() – Adding Numeric Values

`sum()` adds up numeric values in an iterable:

```
sales = [100.50, 230.75, 89.99]
total = sum(sales)
```

This is commonly used for totals (revenue, hours worked, scores, etc.). Note that **`sum()`** expects numeric values; mixing strings or non-numeric types will cause errors.

6.6.3 min() and max() – Extremes in a Collection

`min()` and **`max()`** help you find the smallest and largest values:

```
temperatures = [72, 68, 75, 70]
lowest = min(temperatures)
highest = max(temperatures)
```

These are useful for simple analytics and summaries, such as finding the best score on a test or the highest sale amount for the day.

6.6.4 sorted() – Sorting Without Changing the Original

`sorted()` takes any iterable and returns a new list with elements in sorted order:

```
values = [5, 3, 9, 1]
sorted_values = sorted(values)

print(values)          # original unchanged
print(sorted_values)   # [1, 3, 5, 9]
```

This is different from the list method **`.sort()`**, which sorts in place:

```
values.sort()          # modifies values directly
```

In many cases, **`sorted()`** is safer because it preserves the original data while giving you a sorted version.

You can also pass **`reverse=True`** to sort descending:

```
sorted_desc = sorted(values, reverse=True)
```

6.6.5 any() – At Least One True

`any()` returns **`True`** if at least one element in the iterable is considered **`True`**:

```
flags = [False, False, True]
print(any(flags))   # True
```

This is useful when you want to know if "any problem exists" or if "any condition has been met":

```
errors = [False, user_input_invalid, file_missing]
if any(errors):
    print("There was at least one error.")
```

6.6.6 all() – All Must Be True

`all()` returns **`True`** only if every element is **`True`** (and the collection is not empty):

```
checks = [True, True, True]
print(all(checks))  # True
```

This is helpful when validating that everything is in a good state:

```
password_ok = len(password) >= 8
has_number = any(ch.isdigit() for ch in password)
has_upper = any(ch.isupper() for ch in password)

if all([password_ok, has_number, has_upper]):
    print("Password meets all requirements.")
```

Note: the any() involve using comprehensions for filtering as you will see explained in the next section.

Using **`any()`** and **`all()`** lets you express "some" and "all" logic cleanly, rather than writing long chains of if statements.

6.7 Comprehensions – Compact and Readable Transformations

As your programs grow, you will often find yourself writing loops that:

- build a new list based on an existing one
- filter items according to some condition
- transform values into a different shape

Python provides comprehensions as a concise way to express these patterns. A comprehension combines a loop and optional condition into a single expression. While you can always write the equivalent for loop, comprehensions often end up clearer once you get used to the syntax.

You should think of comprehensions as:
"Take each item x from this collection, maybe filter it, and build a new collection from the results."

6.7.1 List Comprehensions – The Core Idea

Let's start with a simple pattern: building a list of squares.

Traditional loop:

```
squares = []
for n in range(1, 6):
    squares.append(n * n)
```

List comprehension:

```
squares = [n * n for n in range(1, 6)]
```

Both versions produce the same result. The comprehension simply moves the "append" logic into a single expression.

The general pattern is:

```
[new_value_expression for item in iterable]
```

This is often easier to read once your eyes are trained for it.

6.7.2 Adding a Filter – Comprehensions with if

Often you don't want every item, just those that meet a condition. For example, selecting even numbers.

Traditional loop with filter:

```
evens = []
for n in range(20):
    if n % 2 == 0:
        evens.append(n)
```

List comprehension:

```
evens = [n for n in range(20) if n % 2 == 0]
```

Real-world example: selecting only active users' emails:

```
active_emails = [u.email for u in users if u.active]
```

6.7.3 Reducing a value from list

Often you want to reduce or calculate a value from a list. One example would be to total up from values.

Say we have an object that has a price, and we want the total of all priced items. Then even find the average, or whatever else we need to do.

```
class Product:
    def __init__(self, name, price):
        self.name = name
        self.price = price

items = [
    Product("Pen", 1.50),
    Product("Notebook", 4.25),
    Product("Marker", 2.00)
]
total = sum([p.price for p in items]) # 7.75
```

6.7.4 Dictionary Comprehensions

You can also build dictionaries using comprehensions:

```
prices = {"apple": 1.50, "banana": 0.75, "cherry": 2.00}

# apply 7% tax
prices_with_tax = {name: price * 1.07 for name, price in prices.items()}
```

Dictionary comprehensions are especially useful when transforming keyed data.

6.7.5 Set Comprehensions

Set comprehensions build sets using **{}** instead of **[]**.

```
words = ["hi", "hello", "hey", "hi"]
unique_lengths = {len(word) for word in words}
```

This is a compact way to compute a distinct set of values.

6.7.6 When to Use Comprehensions

Comprehensions are not required—you can always use regular loops and append() calls. However, they shine when you want to:

- clearly show that you're building a new collection
- perform simple transformations or filtering in one place
- keep code compact without sacrificing clarity

A good rule of thumb:
If the logic is simple and fits on one readable line → a comprehension is often great.
If the logic is complex or involves many steps → a regular loop is often better.

As you continue in this course, you will see comprehensions frequently. They are one of the "Pythonic" tools that help you write clean, expressive code.

Key Takeaways

- Python offers several core data structures—lists, dictionaries, sets, and tuples—each suited for different kinds of tasks.
- Lists are ordered and mutable, support indexing and slicing, and can store 2D table-like data.
- Dictionaries store data as key–value pairs, allow fast lookups, and support safe retrieval with .get().
- Sets store unique, unordered items and excel at membership checks and operations like union, intersection, and difference.
- Tuples are ordered but immutable, making them ideal for fixed data, returning multiple values, and serving as dictionary keys.
- Constructor functions—list(), dict(), set(), tuple()—let you convert between data types depending on the behavior you need.
- Built-in functions like len(), sum(), min(), max(), sorted(), any(), all() make it easy to analyze and evaluate collections.
- Comprehensions (list, set, dictionary) provide a compact, readable way to transform or filter collections in a single expression.

- Use comprehensions for simple, clear transformations; use loops when logic becomes more complex.
- Understanding these structures and tools helps you write cleaner, more flexible, and more expressive Python code.